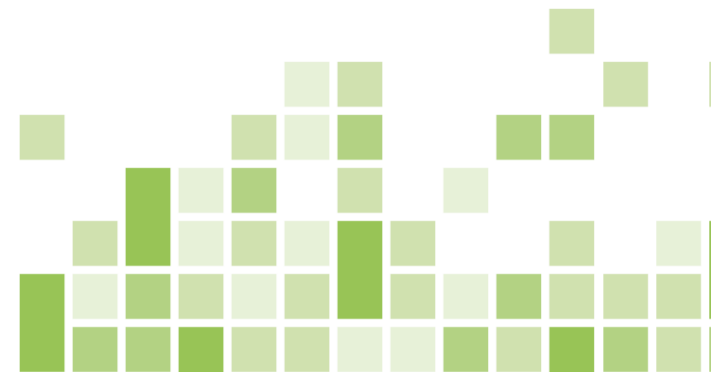




ТОМСКИЙ
ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ



МАТЕМАТИЧЕСКОЕ МОДЕЛИРОВАНИЕ ФИЗИЧЕСКИХ ПРОЦЕССОВ ЛЕКЦИЯ №1

«Введение в курс математического моделирования физических процессов»

Отделение ядерно-топливного цикла

Лектор:
Профессор ОЯТЦ ИЯТШ
Горюнов А.Г.

2024

План лекции

1.1 Введение в курс математического моделирования физических процессов.

1.2 Введение в Matlab, интерфейс программы, редактор Matlab. Язык Matlab: константы и переменные;

1.3 Типы данных: скалярные, символьные, дата, время;

1.4 Операторы цикла и условного перехода;

1.5 Вектора, матрицы и массивы. Индексация в векторах. Создание векторов.

1.6 Введение в Python для задач моделирования.

Литература по курсу:

<https://portal.tpu.ru/SHARED/a/ALEX1479/study/Matmod/Tab>

1.1 Введение в курс математического моделирования физических процессов

Целью данного курса является формирование у обучающихся знаний, умений и приобретение опыта в области математического моделирования процессов и объектов атомной отрасли с использованием стандартных методов и компьютерных кодов для проектирования и анализа.

Цель достигается за счет выполнения следующих **задач**:

- лекции (32 часа), получение информации о теоретическом разделе дисциплины, 16 баллов;
- практические занятия (32 часа), 2 контрольные работы 24 балла;
- лабораторные работы (24 часа), 6 лабораторных работ на 40 баллов;
- Экзамен, 20 баллов.

В дисциплине предусмотрен курсовой проект, диф. зачет, самостоятельная работа 128 часов.

1.1.1 Основные понятия

Моделирование – это научный прием, метод изучения, познания реального окружающего мира.

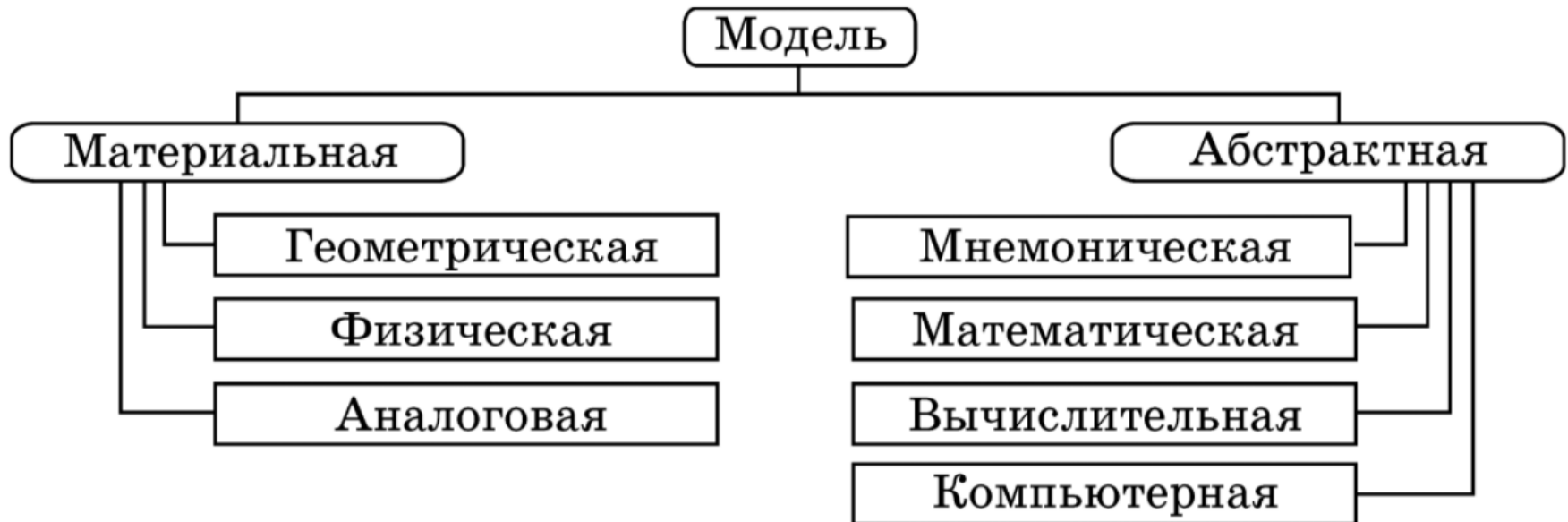
Процесс моделирования включает следующие задачи:

- ✓ реальный изучаемый объект (физическая система, процесс, явление), называемый **оригиналом**, замещается его **моделью** (физическим или абстрактным объектом);
- ✓ модель воспроизводит (имитирует) те свойства и характеристики оригинала, которые существенны для достижения поставленной цели моделирования (для решения конкретной задачи);
- ✓ над моделью проводятся эксперименты и исследования, на основе которых делаются выводы о свойствах объекта – оригинала.

Математическое моделирование предполагает описание исследуемых явлений, процессов, систем различной физической природы языком математических соотношений.

1.1.2 Классификация моделей

Классификация моделей по степени их абстрагирования от оригинала:



Голубева, Н. В. Математическое моделирование систем и процессов : учебное пособие / Н. В. Голубева. — 2-е изд., стер. — Санкт-Петербург : Лань, 2016. — 192 с.— Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/76825> (дата обращения: 03.03.2020). — Режим доступа: для авториз. Пользователей.

1.1.3 Классификация математических моделей

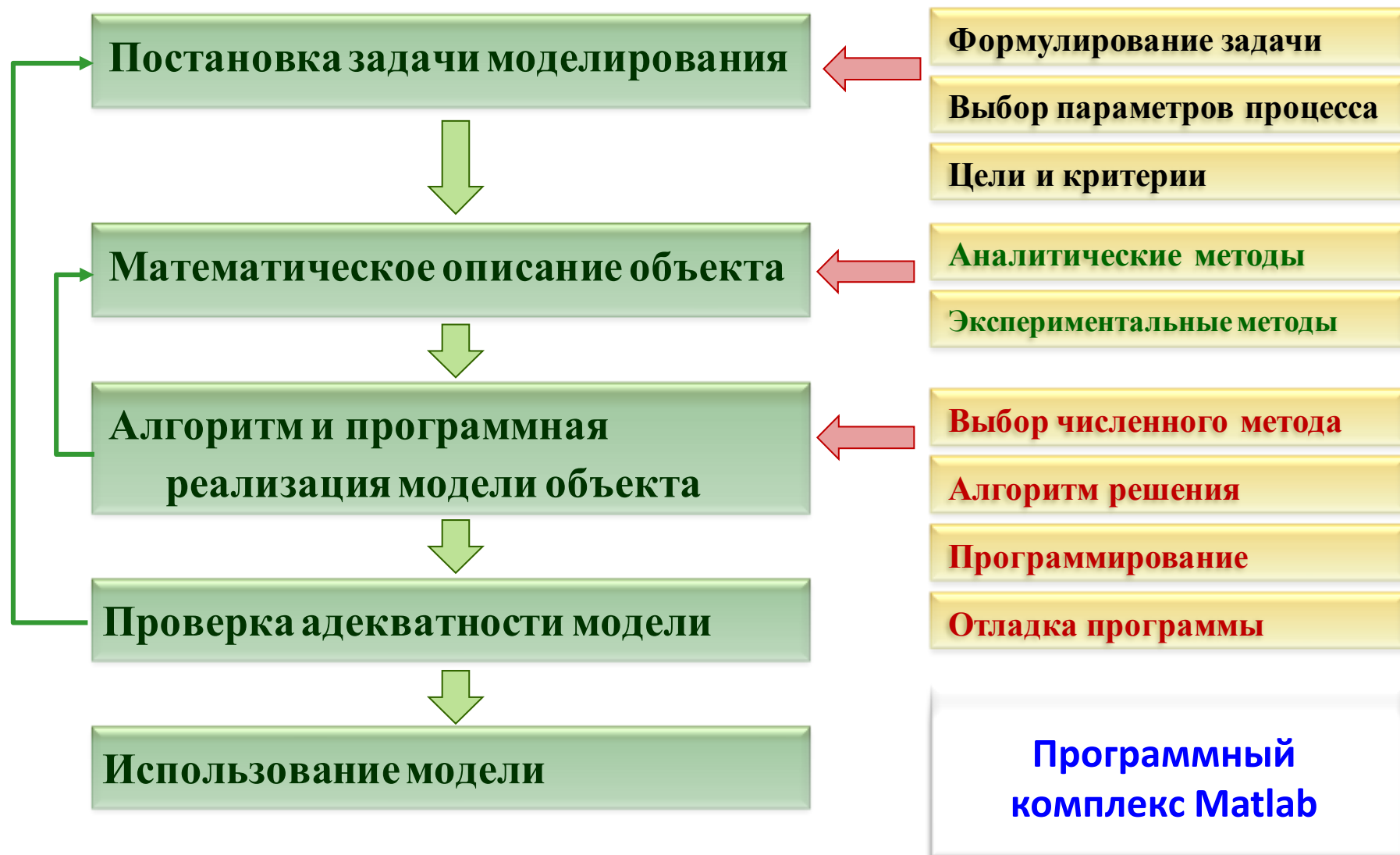
Признак классификации	Математические модели
Характер отображаемых свойств объекта	Структурные; функциональные
Принадлежность к иерархическому уровню	Микроуровня; макроуровня; метауровня
Степень детализации описания внутри одного уровня	Полные; макромодели
Способ представления свойств объекта	Аналитические, алгоритмические, имитационные
Способ получения модели	Теоретические, эмпирические

Аналитическая модель – математическая **модель**, представляющая собой совокупность **аналитических** выражений и зависимостей, позволяющих оценивать те или иные свойства моделируемого объекта.

Имитационная модель – универсальное средство исследования сложных систем, представляющее собой логико-алгоритмическое описание поведения отдельных элементов системы и правил их взаимодействия, отображающих последовательность событий, возникающих в моделируемой системе.

1.1.4 Подход к математическому моделированию процессов

7



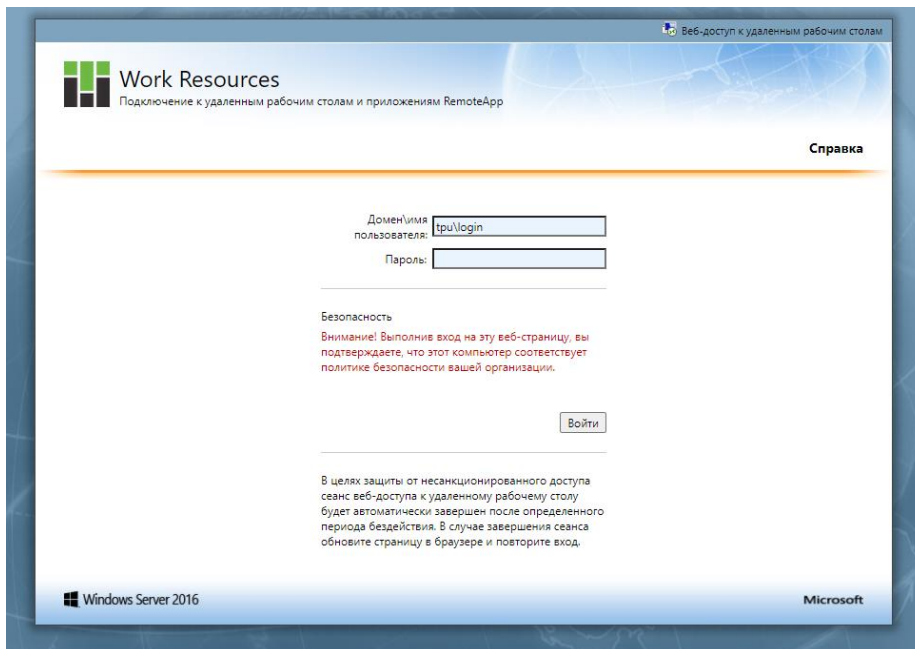
1.2 Введение в Matlab, интерфейс программы, редактор Matlab. Язык Matlab: константы и переменные

1.2.1 Работа с Matlab в ТПУ

Сотрудники ТПУ и студенты имеют возможность работать с Matlab на серверах ТПУ. Для этого необходим компьютер с доступом в интернет.

Для запуска Matlab в любом браузере необходимо набрать адрес:

<http://www.vap.tpu.ru/>



В поле «имя пользователя» необходимо ввести **TPU\login**, где login – Ваш корпоративный логин (логин для входа в почту, личный кабинет и т.д.)

1.2.2 Запуск Matlab

Необходимо перейти в папку «Математические»

Веб-доступ к удаленным рабочим столам

 **Work Resources**
Подключение к удаленным рабочим столам и приложениям RemoteApp

Удаленные приложения RemoteApp и рабочие столы | [Справка](#) | [Выйти](#)

Текущая папка: /

 1C	 Adobe	 DiaLUX	 Labs Moodle	 Microsoft Office	 Графические	 Замена MS Visio	 ИНЭО	 ИШПР	 Математиче...
 Моделиров...	 Работа с БД	 Разработка приложений	 CAE	 САПР	 Химические	 Электроника	 chrome	 ПОУП	 Учет контингента студентов

Windows Server 2022

Microsoft

1.2.2 Запуск Matlab

Запустить Matlab используя ярлык в папке.

Веб-доступ к удаленным рабочим столам

Work Resources
Подключение к удаленным рабочим столам и приложениям RemoteApp

Удаленные приложения RemoteApp и рабочие столы | Справка | Выйти

Текущая папка: /Математические

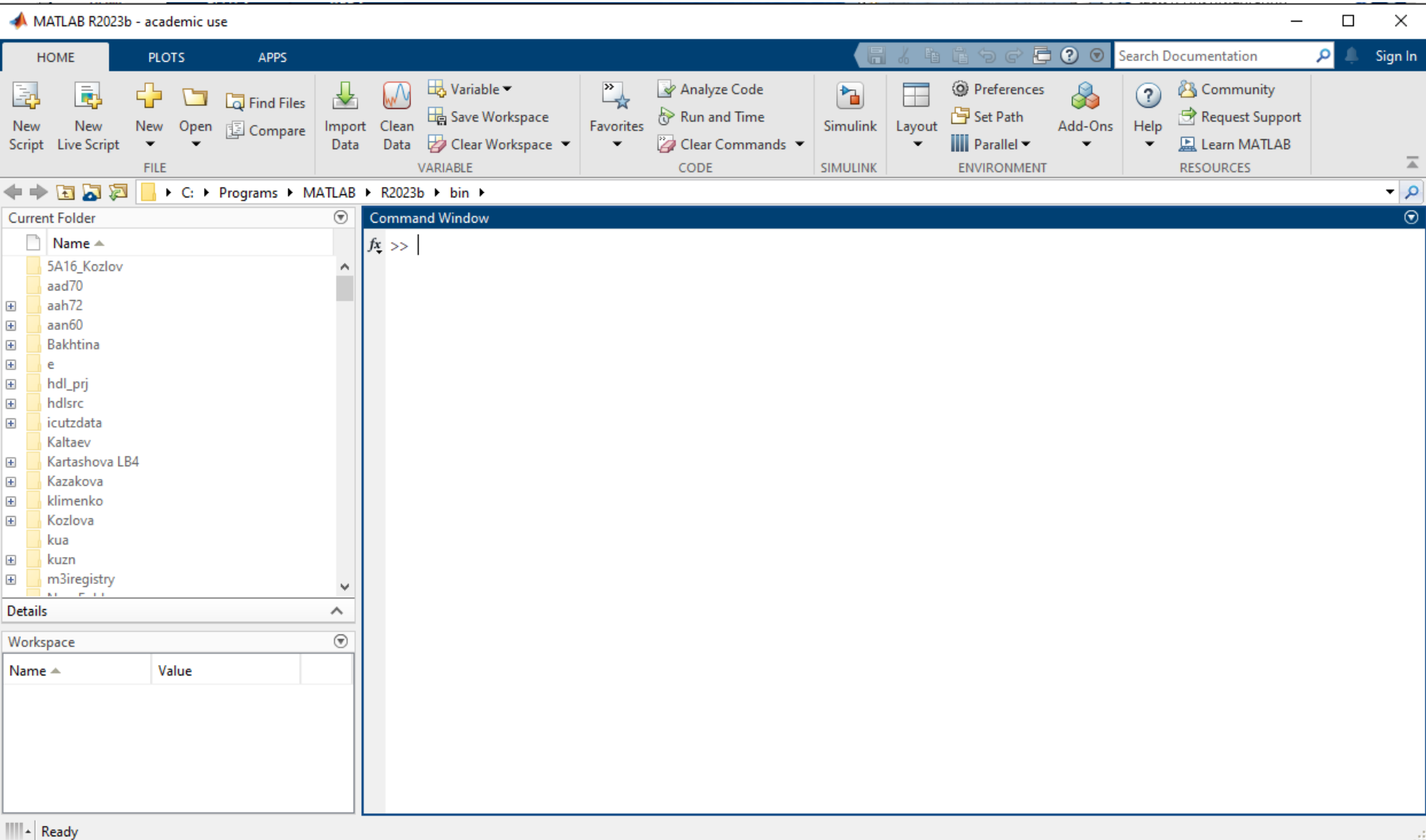
Наверх	COMSOL Multiphysics 6.2 (Classkit)	Maple 18	Mathcad 15-сервер1	Mathcad 15-сервер2	MATLAB R2023b-сервер 3	MATLAB R2023b-сервер1	MATLAB R2023b-сервер2	Origin 2023	Prime 6
Prime 7	PTC Mathcad Prime 8	scilab-2024 (64-bit) Desktop	STATISTICA 13	Wolfram Mathematica 12.2	Wolfram Mathematica 14	Оптика	Оптика		

Windows Server 2022

Microsoft

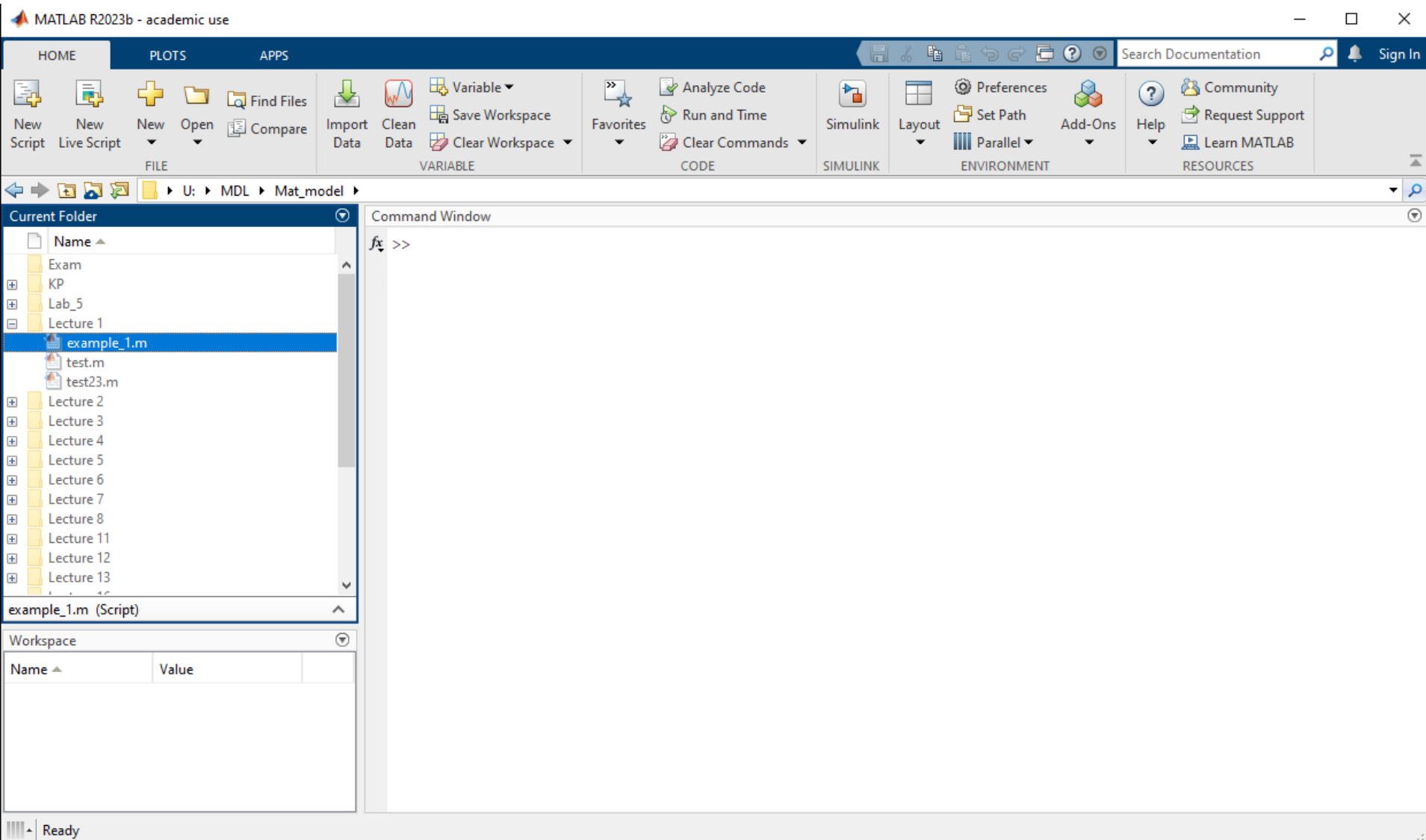
1.2.3 Интерфейс Matlab

11



1.2.3 Интерфейс Matlab

Необходимо выбрать Ваш диск и директорию для сохранения файлов



1.2.4 Константы в Matlab

Константа – это предварительно определенное числовое или символьное значение, представленное уникальным именем. Числа (например 1, -2 и 1.23) являются безымянными **числовыми константами**.

Другие виды констант в MATLAB принято назвать **системными переменными**, поскольку, с одной стороны, они задаются системой при ее загрузке, а с другой – могут переопределяться. Основные системные переменные, применяемые в системе MATLAB, указаны ниже:

i или **j** – мнимая единица (корень квадратный из -1);

pi – число **п** – 3.1415926...;

eps – погрешность операций над числами с плавающей точкой (2^{-52});

realmin – наименьшее число с плавающей точкой (2^{-1022});

realmax – наибольшее число с плавающей точкой (2^{1023});

inf – значение машинной бесконечности;

ans – переменная, хранящая результат последней операции и обычно вызывающая его отображение на экране дисплея;

NaN – указание на нечисловой характер данных (**Not-a-Number**).

1.2.4 Константы в Matlab

14

The image shows the MATLAB R2013a interface. The top menu bar includes HOME, PLOTS, and APPS. Below it is a toolbar with icons for New Script, New, Open, Find Files, Compare, Import Data, Save Workspace, Open Variable, Clear Workspace, Analyze Code, Run and Time, Clear Commands, Simulink Library, Layout, Set Path, Parallel, Help, Community, Request Support, and Add-Ons. The main workspace is divided into three panes: Current Folder, Command Window, and Workspace.

Current Folder: Shows the file structure of the current folder, including 'Matlab', 'Multisym', 'model.asv', and 'model.m*'. The 'Multisym' folder is selected.

Command Window: Displays the results of MATLAB commands entered in the Command Window. The commands and their outputs are as follows:

```
>> 2*pi
ans =
    6.2832

>> eps
ans =
    2.2204e-16

>> realmax
ans =
    1.7977e+308

>> 1/0
ans =
    Inf

>> Inf
ans =
    Inf

>> 0/0
ans =
    NaN
```

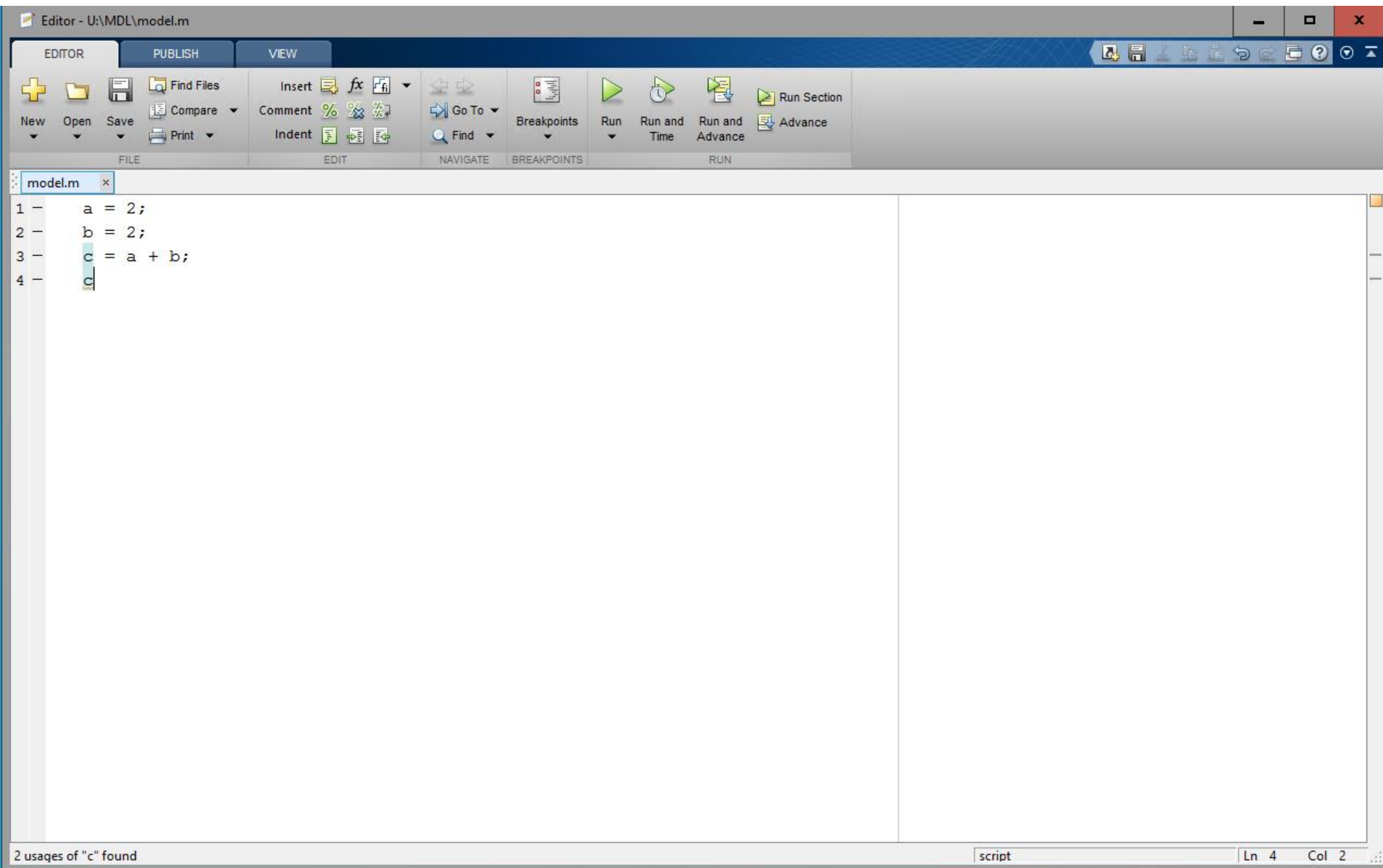
Workspace: Displays the variables currently in the workspace. The variables and their values are:

Name	Value
a	2
ans	NaN
b	2
c	4

Command History: Shows the list of commands entered in the Command Window, including 'My fist model', 'Untitled2', 'model', '2*pi', 'eps', 'realmax', '1/0', 'Inf', and '0/0'.

1.2.5 Редактор языка Matlab.

Переменные в Matlab



1.3 Типы данных в Matlab

1.3.1 Основные типы данных

Основные типы данных

Основные типы данных: числовые, logical , char , cell , struct , table и function_handle .

Command Window

```
>> a = 1.14;  
>> b = [1 2 3];  
>> c = '123';  
>> whos
```

Name	Size	Bytes	Class	Attributes
a	1x1	8	double	
ans	1x1	8	double	
b	1x3	24	double	
c	1x3	6	char	

fx >>

Command Window

```
>> a = 1.1479; % По умолчанию двойная точность  
b = single(a); % В переменной b получим одинарную точность  
>> whos
```

Name	Size	Bytes	Class	Attributes
a	1x1	8	double	
b	1x1	4	single	

1.3.1 Основные типы данных

```
Command Window
>> a = int32(10); % целая переменная 32 разряда
b = int8(127); % целая переменная 8 bit
>> whos
```

Name	Size	Bytes	Class	Attributes
a	1x1	4	int32	
b	1x1	1	int8	

```
>> c = logical(1); % логическая переменная true / false (1 или 0)
>> whos
```

Name	Size	Bytes	Class	Attributes
a	1x1	4	int32	
b	1x1	1	int8	
c	1x1	1	logical	

1.3.2 Символьные переменные

```

Command Window
>> x = sym('x'); % создаем символьную переменную
>> f1 = (x^100 - 2*x + 1)/(x^10 + 2*x + 1); % создали функцию
>> pretty(f1) % вывести функцию "красиво"

      100
      x      - 2 x + 1
      -----
      10
      x      + 2 x + 1

```

1.3.3 Форматы представления даты

Dateform(номер)	Dateform (строка)	Пример
0	'dd-rmiM-yyyy HH:MM:SS'	11-Mar-1995 03:45
1	'dd-mmM-yyyy'	01-Mar-1995
2	'mm/dd/yy'	03/01/95
3	' mmm '	Mar
4	'm'	M
5	'mm'	3
6	'mm/dd'	03/01
7	'dd'	1
8	'ddd'	Wed
9	'd'	W
10	'yyyy'	1995
11	'yy'	95
12	' mmm yy '	Mar95
13	'HH:MM:SS'	15:45:17

1.3.3 Форматы представления даты

Command Window

```
>> n = datevec('09/03/20') % перевод из типа "дата" в числовой массив
```

```
n =
```

```
Columns 1 through 5
```

```
2020
```

```
9
```

```
3
```

```
0
```

```
0
```

1.3.4 Форматы представления времени

Command Window

```
>> t = clock; % clock — возвращает вектор из 6 элементов, содержащий текущую дату и время
```

```
>> % в десятичной форме [год месяц день час минуты секунды].
```

```
>> % Первые пять элементов этого вектора — целые числа. Шестой элемент имеет
```

```
>> %несколько десятичных знаков после запятой.
```

```
>> t
```

```
t =
```

```
1.0e+03 *
```

```
2.0200
```

```
0.0090
```

```
0.0020
```

```
0.0190
```

```
0.0380
```

```
0.0287
```

```
>> time = fix(clock); % функция fix(clock) округляет число секунд до целого значения.
```

```
>> time
```

```
time =
```

```
2020
```

```
9
```

```
2
```

```
19
```

```
40
```

```
41
```

1.4 Операторы цикла и условного перехода

1.4.1 Условный оператор if

```
if <выражение>  
<операторы1> % выполняются, если истинно  
условие  
else  
<операторы2> % выполняются, если условие  
ложно  
end
```

```
if <выражение1>  
<операторы1> % выполняются, если истинно  
выражение1  
elseif <выражение2>  
<операторы2> % выполняются, если истинно  
выражение2  
...  
elseif <выражениеN>  
<операторыN> % выполняются, если истинно  
выражениеN  
end
```

1.4.1 Условный оператор if

Command Window

```
>> x = 5;  
if x > 0  
disp(1); % выполняется, если x > 0  
elseif x < 0  
disp(-1); % выполняется, если x < 0  
else  
disp(0); % выполняется, если x = 0  
end  
1
```

Command Window

```
>> x = -5;  
if x > 0  
disp(1); % выполняется, если x > 0  
elseif x < 0  
disp(-1); % выполняется, если x < 0  
else  
disp(0); % выполняется, если x = 0  
end  
-1
```

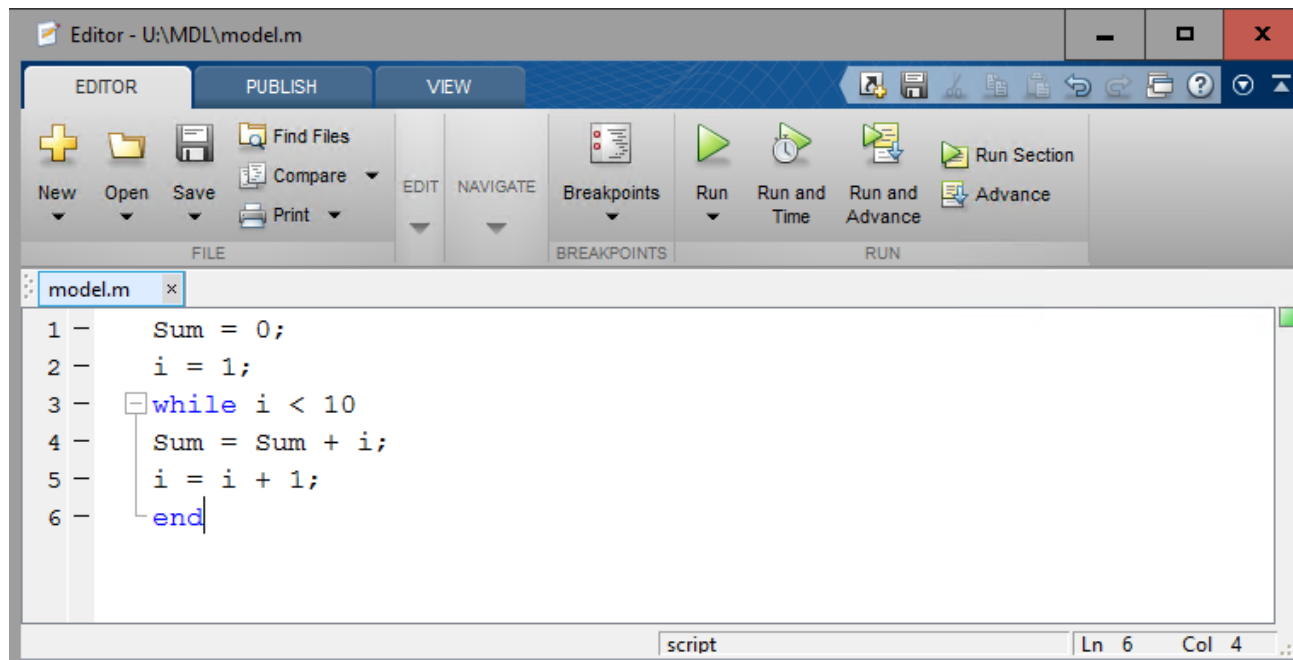
Command Window

```
>> x = 0;  
if x > 0  
disp(1); % выполняется, если x > 0  
elseif x < 0  
disp(-1); % выполняется, если x < 0  
else  
disp(0); % выполняется, если x = 0  
end  
0
```

1.4.2 Оператор цикла while

```
while <условие>  
<операторы>  
end
```

```
Command Window  
  
>> Sum = 0;  
>> i = 1;  
>> while i < 10  
Sum = Sum + i;  
i = i + 1;  
end  
>> Sum  
  
Sum =  
  
45
```



1.5 Вектора, матрицы и массивы.

Индексация в векторах. Создание векторов.

1.5.1 Вектора в Matlab

Command Window

```
>> a = [1 2 3 4];           % вектор-строка
>> disp( a(1) );           % отображение значения 1-го элемента вектора
disp( a(2) );               % отображение значения 2-го элемента вектора
disp( a(3) );               % отображение значения 3-го элемента вектора
disp( a(4) );               % отображение значения 4-го элемента вектора
1
2
3
4
```

Command Window

```
>> a = [10; 20; 30; 40];    % вектор-столбец
>> c = a(1) + a(4)
```

c =

50

1.5.2 Матрицы в Matlab

Command Window

```
>> E = [1 0 0; 0 1 0; 0 0 1];           % единичная матрица 3x3
```

```
>> E
```

```
E =
```

```
    1    0    0
    0    1    0
    0    0    1
```

```
>> A = [1 0 0
        0 1 0
        0 0 1] % единичная матрица 3x3
```

```
A =
```

```
    1    0    0
    0    1    0
    0    0    1
```

1.5.3 Индексация в векторах и матрицах

Command Window

```
>> A = [1 2 3  
        4 5 6  
        7 8 9];
```

```
>> A(1,1)
```

```
ans =
```

```
1
```

```
>> c = A(1,1) + A(1,2) + A(3,3)
```

```
c =
```

```
12
```

```
>> A(1,2)
```

```
ans =
```

```
2
```

```
>> A(3,3)
```

```
ans =
```

```
9
```

1.6 Введение в Python для задач моделирования.

1.6.1 Установка Python

Установка Python в Linux:

```
apt-get update  
apt-get upgrade python3  
или  
apt-get install python3
```

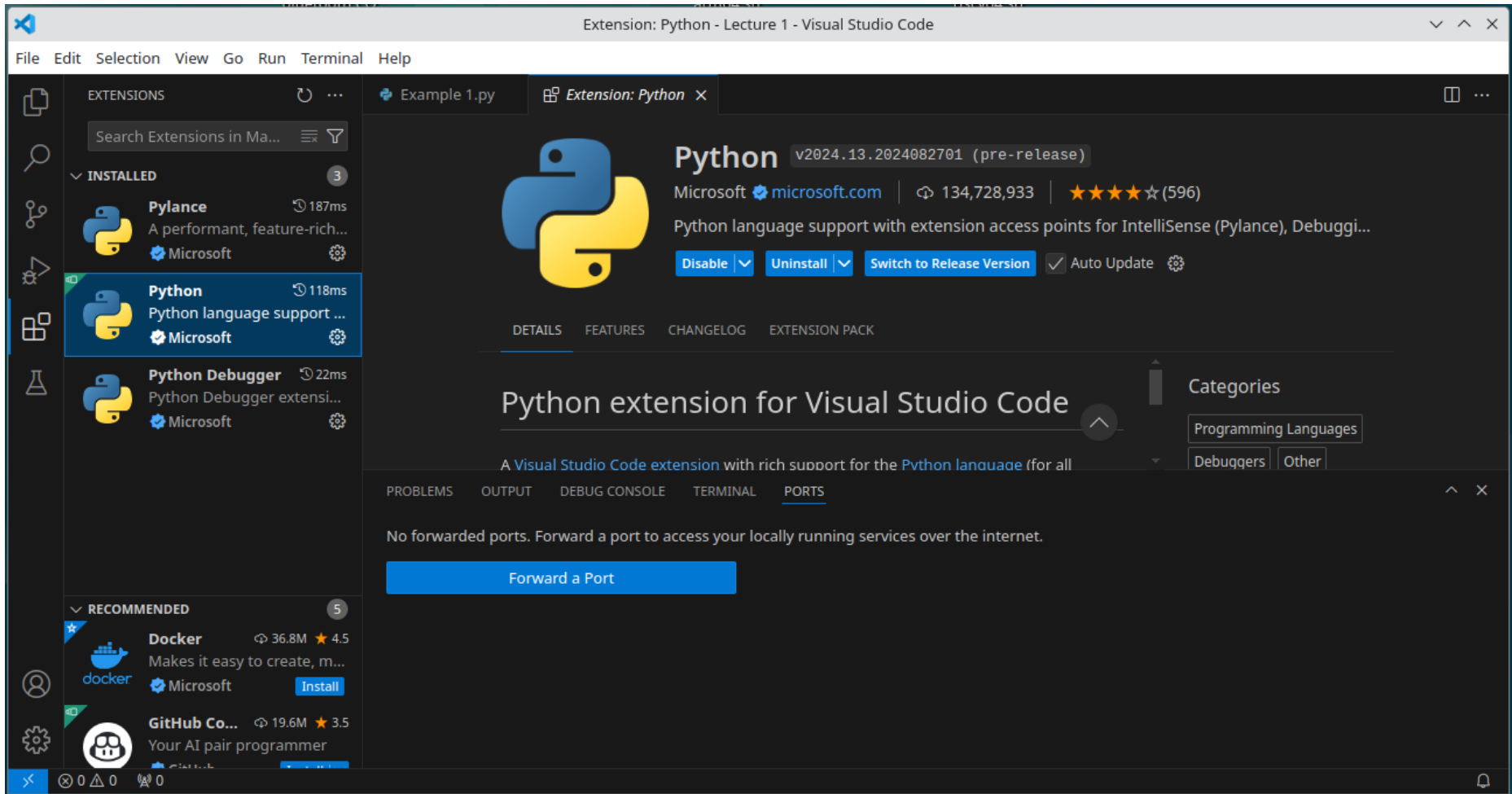
Установка Python в Windows:

<https://www.python.org/downloads/windows/>

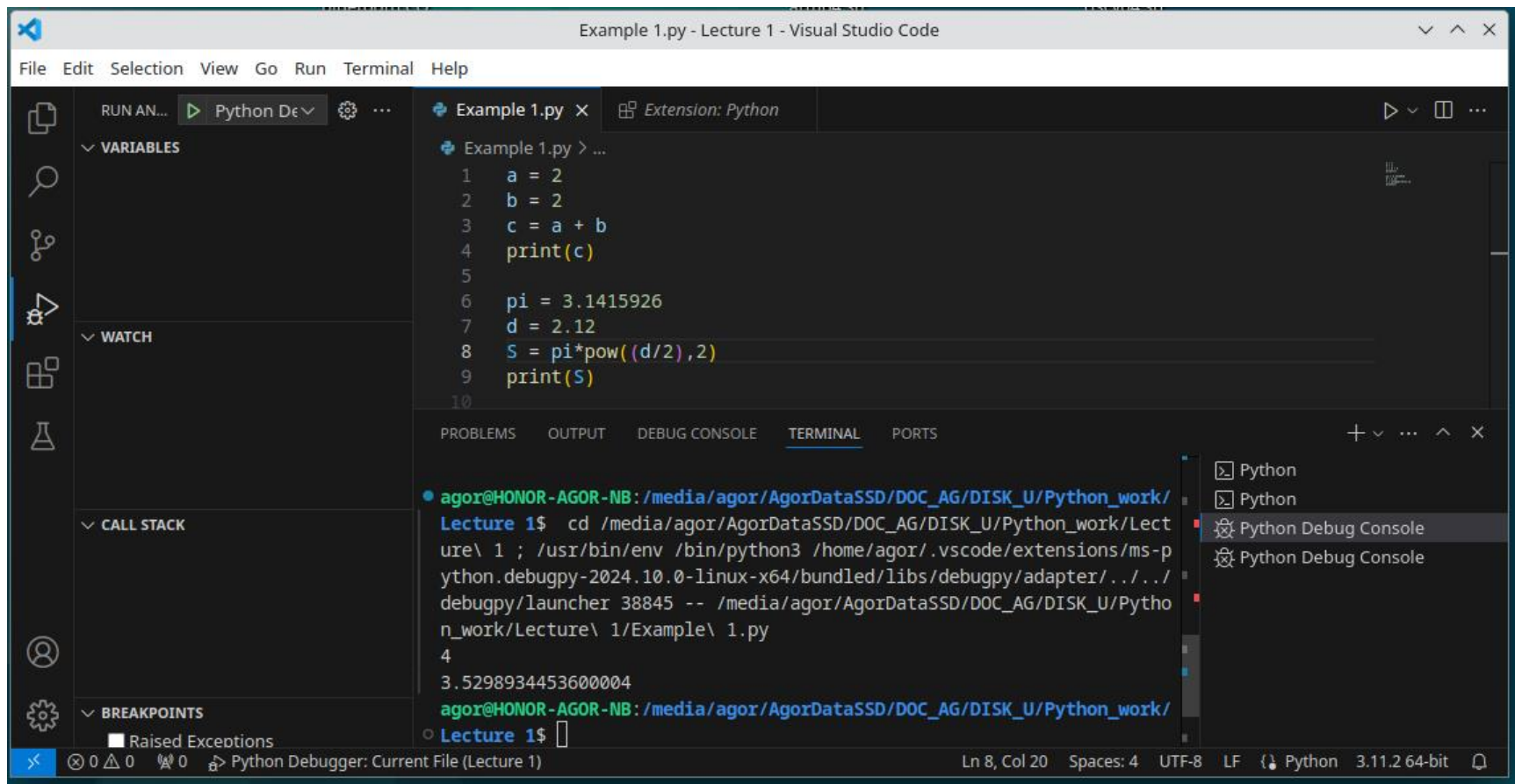
Установка Visual Studio Code:

<https://code.visualstudio.com/>

1.6.2 Настройка Visual Studio code для Python



1.6.3 Основные типы данных Python



Три основные типа переменных (чисел) в Python:

```
x = 1 # int
y = 2.8 # float
z = 1j # complex
```

1.6.3 Основные типы данных Python

Примеры преобразования типов в Python:

```
x = int(1) # x станет 1  
y = int(2.8) # y станет 2  
z = int("3") # z станет 3
```

```
x = float(1) # x станет 1.0  
y = float(2.8) # y станет 2.8  
z = float("3") # z станет 3.0  
w = float("4.2") # w станет 4.2
```

```
x = str("s1") # x станет 's1'  
y = str(2) # y станет '2'  
z = str(3.0) # z станет '3.0'
```

1.6.4 Операторы в Python

Арифметические операторы используются с числовыми значениями для выполнения общих математических операций:

Оператор	Значение	Пример
+	добавление	4 + 5
—	вычитание	8 — 3
*	умножение	5 * 5
/	деление	4 / 2
%	остаток от деления	7 % 2
**	возведение в степень	2 ** 3
//	целочисленное деление	15 // 4

<https://pythonru.com/uroki/operatory-v-python-uroki-po-python-dlja-nachinajushhih>

1.6.4 Операторы в Python

Операторы присваивания используются для присваивания значений переменным:

Оператор	Пример	Так же как
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x — 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

1.6.4 Операторы в Python

Операторы сравнения используются для сравнения двух значений:

Оператор	Значение	Пример
<code>==</code>	равно	<code>x == y</code>
<code>!=</code>	не равно	<code>x != y</code>
<code>></code>	больше чем	<code>x > y</code>
<code><</code>	меньше чем	<code>x < y</code>
<code>>=</code>	больше чем или равно	<code>x >= y</code>
<code><=</code>	меньше чем или равно	<code>x <= y</code>

Логические операторы используются для объединения условных операторов:

Оператор	Значение	Пример
<code>and</code>	Возвращает значение True если оба утверждения верны	<code>x < 5 and x < 10</code>
<code>or</code>	Возвращает True если одно из утверждений верно	<code>x < 5 or x < 4</code>
<code>not</code>	Меняет результат, возвращает False если результат True	<code>not(x < 5 and x < 10)</code>

1.6.5 Условный оператор if в Python

```
if condition:  
    <indented statement 1>  
    <indented statement 2>  
  
<non-indented statement>
```

Example 2.py X

Example 2.py > ...

```
1  a = 6  
2  b = 3  
3  if a > b:  
4      print("a > b")  
5  else:  
6      print("a <= b")  
7  
8  c = 1  
9  if c >= 10:  
10     print(a)  
11 else:  
12     print(b)  
13
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
agor@HONOR-AGOR-NB: /media/agor/AgorDataSSD/DOC_AG/DISK_U/Python_work/Lecture 1$ cd /media/agor/AgorDataSSD/DOC_AG/DISK_U/Python_work/Lecture 1 ; /usr/bin/env /bin/python3 /home/agor/.vscode/extensions/ms-python.debugpy-2024.10.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 48675 -- /media/agor/AgorDataSSD/DOC_AG/DISK_U/Python_work/Lecture 1/Example 2.py  
a > b  
3
```

1.6.6 Циклы: оператор for

```
for <переменная> in <последовательность>:  
    <действие>  
else:  
    <действие>
```

Example 3.py X

Example 3.py > ...

```
1  a = 0  
2  for i in range(1,10):  
3      a+= 1  
4  else:  
5      print("конец цикла")  
6  
7  print(a)  
8  
9  
10
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

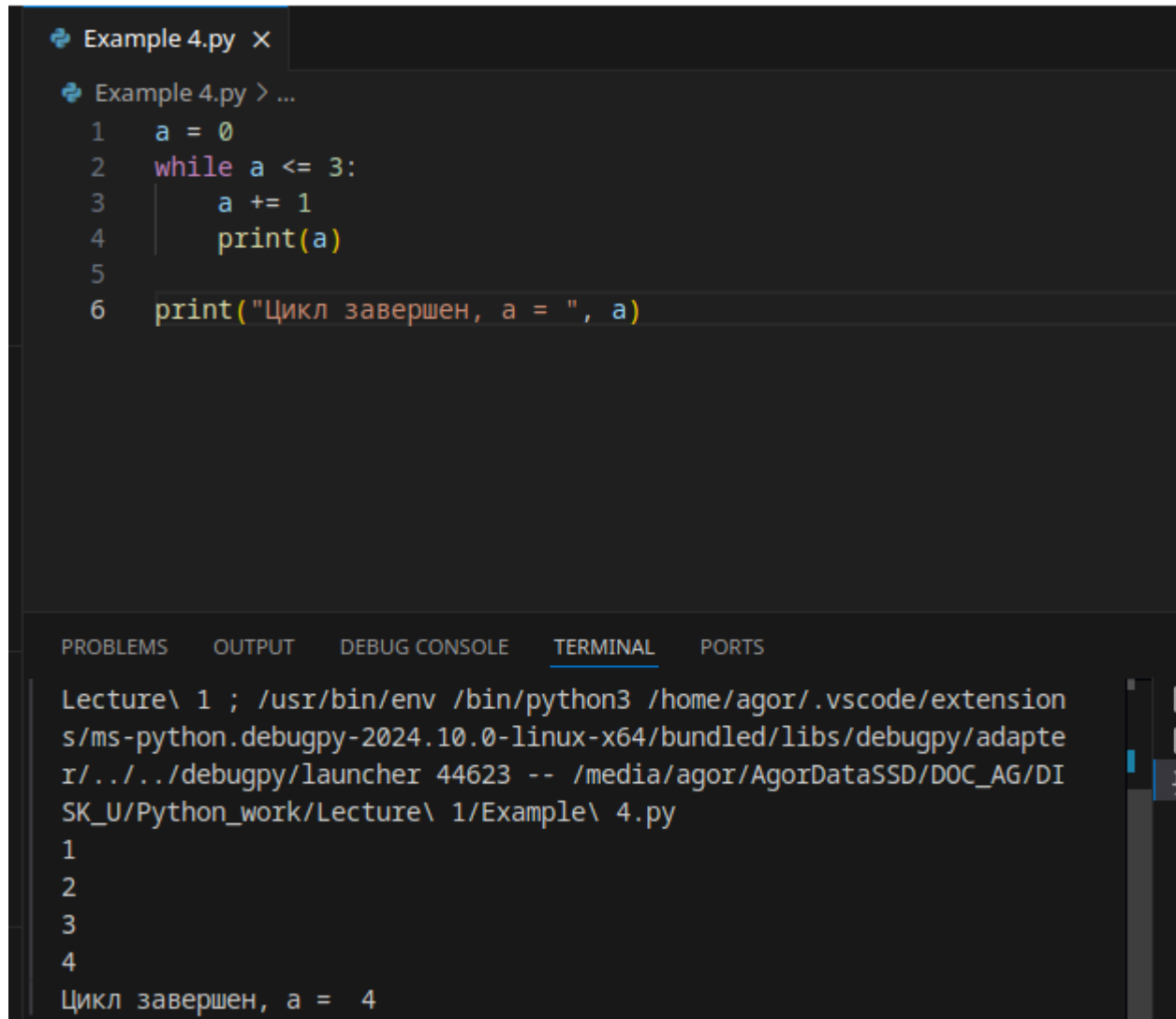
TERMINAL

PORTS

```
agor@HONOR-AGOR-NB: /media/agor/AgorDataSSD/DOC_AG/DISK_U/Python_work  
• k/Lecture 1$ cd /media/agor/AgorDataSSD/DOC_AG/DISK_U/Python_work/  
Lecture\ 1 ; /usr/bin/env /bin/python3 /home/agor/.vscode/extension  
s/ms-python.debugpy-2024.10.0-linux-x64/bundled/libs/debugpy/adapte  
r/../../../../debugpy/launcher 35075 -- /media/agor/AgorDataSSD/DOC_AG/DI  
SK_U/Python_work/Lecture\ 1/Example\ 3.py  
конец цикла  
9
```

1.6.7 Циклы: оператор while

```
while test_expression:  
    body of while
```



The screenshot shows a VS Code editor window with a file named 'Example 4.py'. The code in the editor is a Python script that initializes a variable 'a' to 0 and enters a 'while' loop that runs as long as 'a' is less than or equal to 3. Inside the loop, 'a' is incremented by 1 and its value is printed. After the loop, a final print statement shows the value of 'a'.

```
Example 4.py X  
Example 4.py > ...  
1 a = 0  
2 while a <= 3:  
3     a += 1  
4     print(a)  
5  
6 print("Цикл завершен, a = ", a)
```

Below the editor, the 'TERMINAL' panel shows the command used to run the script and its output. The command is executed from a directory path, and the output shows the first four iterations of the loop, with the final line indicating the loop is complete and 'a' equals 4.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
Lecture\ 1 ; /usr/bin/env /bin/python3 /home/agor/.vscode/extension  
s/ms-python.debugpy-2024.10.0-linux-x64/bundled/libs/debugpy/adapte  
r/../../debugpy/launcher 44623 -- /media/agor/AgorDataSSD/DOC_AG/DI  
SK_U/Python_work/Lecture\ 1/Example\ 4.py  
1  
2  
3  
4  
Цикл завершен, a = 4
```

1.6.8 Матрицы в Python

```
Example 5.py ×  
Example 5.py > ...  
1  a = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
2  print(a)  
3  
4  import numpy as np  
5  A = np.array([[1,2,3],[4,5,6],[7,8,9]])  
6  print("Матрица с использованием библиотеки NumPy")  
7  print(A)  
8  
  
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
Lecture\ 1 ; /usr/bin/env /bin/python3 /home/agor/.vscode/extensions/ms-python.debugpy-2024.10.0-linux-x64/bundled/libs/debugpy/adapted/../../debugpy/launcher 35103 -- /media/agor/AgorDataSSD/DOC_AG/DISK_U/Python_work/Lecture\ 1/Example\ 5.py  
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
Матрица с использованием библиотеки NumPy  
[[1 2 3]  
 [4 5 6]  
 [7 8 9]]
```

1.6.9 Индексация в матрицах

```
Example 6.py x
Example 6.py > ...
1  a = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
2  print(a)
3  c = a[0][0] + a[1][1] + a[2][2]
4  print(c)
5
6  import numpy as np
7  B = np.array([[1,2,3],[4,5,6],[7,8,9]])
8  print(B)
9  c = B[0][0] + B[1][1] + B[2][2]
10 print(c)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
ns/ms-python.debugpy-2024.10.0-linux-x64/bundled/libs/debugpy/adapt
er/../../debugpy/launcher 52631 -- /media/agor/AgorDataSSD/DOC_AG/D
ISK_U/Python_work/Lecture\ 1/Example\ 6.py
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
15
[[1 2 3]
 [4 5 6]
 [7 8 9]]
15
```